Traffic and Accident Management System using Computer Vision

Course Code – INT345
Course Name – Computer Vision

Names – Shahid Ansari
Roll No – 20
Reg No - 12321439

Prepared for
Continuous Assessment 3
Spring 2025

Table of Contents

REVISION HISTORY ..... ERROR! BOOKMARK NOT DEFINED.

1. INTRODUCTION **1**

1.1 PURPOSE 1
1.2 SCOPE 1
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS 2
1.4 REFERENCES 2
1.5 OVERVIEW 3

2. GENERAL DESCRIPTION **3**

2.1 PRODUCT PERSPECTIVE 3
2.2 PRODUCT FUNCTIONS 3
2.3 USER CHARACTERISTICS ..... **ERROR! BOOKMARK NOT DEFINED.**
2.4 GENERAL CONSTRAINTS 4
2.5 ASSUMPTIONS AND DEPENDENCIES ..... **ERROR! BOOKMARK NOT DEFINED.**

3. SPECIFIC REQUIREMENTS **5**

3.1 EXTERNAL INTERFACE REQUIREMENTS 6
3.1.1 User Interfaces *6*
3.1.2 Hardware Interfaces *7*
3.1.3 Software Interfaces *7*
3.1.4 Communications Interfaces *8*
3.2 FUNCTIONAL REQUIREMENTS 8
3.2.1 <Functional Requirement or Feature #1> ..... ***Error! Bookmark not defined.***
3.2.2 <Functional Requirement or Feature #2> *9*
3.5 NON-FUNCTIONAL REQUIREMENTS 3
3.5.1 Performance *3*
3.5.2 Reliability *3*
3.5.3 Availability *3*
3.5.4 Security *3*
3.5.5 Maintainability *3*
3.5.6 Portability *3*
3.7 DESIGN CONSTRAINTS 3
3.9 OTHER REQUIREMENTS 3

4. ANALYSIS MODELS ..... ERROR! BOOKMARK NOT DEFINED.

4.1 DATA FLOW DIAGRAMS (DFD) 4

5. GITHUB LINK **5**

6. DEPLOYED LINK **6**

7. CLIENT APPROVAL PROOF **7**

8. CLIENT LOCATION PROOF **8**
9. TRANSACTION ID PROOF **9**
10. EMAIL ACKNOWLEDGEMENT **10**
11. GST No **11**

A. APPENDICES .....

A.1 APPENDIX 1 .....
A.2 APPENDIX 2 .....

1. Introduction

With the increasing number of vehicles on roads and highways, managing traffic efficiently and ensuring road safety has become a major challenge. Traditional traffic monitoring methods rely heavily on manual observation, which is time-consuming, less accurate, and not suitable for real-time decision-making during critical situations such as accidents.

To address these issues, this project proposes a Traffic and Accident Management System using Computer Vision. The system uses video footage to automatically detect vehicles, track their movement, estimate their speed, and identify abnormal behavior such as sudden stopping or collisions that may indicate accidents.

The system applies deep learning techniques for object detection and tracking, making it capable of handling multiple vehicles simultaneously. This project demonstrates the practical application of machine learning in traffic analysis and contributes toward intelligent transportation systems and smart city solutions.

1.1 Purpose

The purpose of this Software Requirement Specification (SRS) document is to define the functional and non-functional requirements of the Traffic and Accident Management System using Computer Vision. It provides a clear understanding of the system's design, features, and performance expectations.

1.2 Scope

The software product to be developed is the Traffic and Accident Management System using Computer Vision. This system is designed as a video analytics application that processes traffic footage to automatically monitor vehicle movement and detect potential accidents.

The system performs functions such as detecting vehicles including cars, buses, trucks, and motorcycles, tracking them across multiple frames using unique identifiers, estimating their speed based on motion analysis, and identifying possible accidents through abnormal behavior such as sudden stopping or collisions. The system generates an annotated output video displaying bounding boxes, vehicle IDs, speed information, and accident alerts.

The system does not include physical hardware components such as cameras or sensors, and it is not integrated with emergency response services. It focuses only on software-based analysis and visualization.

The application of this system lies in intelligent traffic monitoring and smart city solutions. The main objective is to reduce manual effort, improve monitoring accuracy, and enable automated detection of traffic incidents. The system aims to provide efficient video processing, accurate vehicle detection, reliable tracking, and effective accident detection with minimal false positives.

1.3 Definitions, Acronyms, and Abbreviations

This subsection provides the definitions of important terms, acronyms, and abbreviations used in this document. These definitions help in clearly understanding the system and ensure that all technical terms are interpreted correctly throughout the SRS.

- Computer Vision (CV): A field of artificial intelligence that enables computers to analyze and understand visual data from images and videos.
- YOLO (You Only Look Once): A deep learning-based object detection algorithm used for detecting vehicles in real-time video frames.
- Object Detection: The process of identifying and locating objects in an image or video using bounding boxes.
- Tracking: The process of assigning a unique ID to each detected object and maintaining its identity across consecutive frames.
- FPS (Frames Per Second): The number of frames processed per second in a video, which affects system performance and speed.
- IoU (Intersection over Union): A metric used to measure the overlap between two bounding boxes, commonly used for collision detection and evaluation.
- Bounding Box: A rectangular box drawn around a detected object to indicate its position in a frame.
- Pixel-to-Meter Conversion: A scaling factor used to convert pixel distance into real-world distance for speed estimation.
- Speed Estimation: The process of calculating the speed of a vehicle based on its movement across frames.
- Accident Detection: The process of identifying abnormal vehicle behavior such as sudden stopping, erratic motion, or collisions.
- Deep Learning: A subset of machine learning that uses neural networks with multiple layers to learn patterns from data.

1.4 References

This section lists the documents and resources used during the development of this project. These references help in understanding the concepts, tools, and techniques used in the system.

- Ultralytics YOLO Documentation, *YOLOv8 Official Documentation*, 2023, Ultralytics. Available at: https://docs.ultralytics.com
- OpenCV Documentation, *Open Source Computer Vision Library*, 2023, OpenCV. Available at: https://docs.opencv.org
- Redmon, J., Farhadi, A., *YOLOv3: An Incremental Improvement*, 2018, arXiv. Available at: https://arxiv.org/abs/1804.02767
- Bewley, A. et al., *Simple Online and Realtime Tracking (SORT)*, 2016, IEEE. Available at: https://arxiv.org/abs/1602.00763
- Zhang, Y. et al., *ByteTrack: Multi-Object Tracking*, 2022, ECCV. Available at: https://arxiv.org/abs/2110.06864
- Python Software Foundation, *Python Documentation*, 2023. Available at: https://docs.python.org

1.5 Overview

This section describes what the rest of this Software Requirement Specification (SRS) document contains and how it is organized.

Section 2 provides the general description of the system, including its overall perspective, main functions, user characteristics, constraints, and assumptions. Section 3 presents the specific requirements of the system, which include external interface requirements, functional requirements, and non-functional requirements. Section 4 explains the analysis model, which shows how data flows through the system.

The remaining sections include project-related information such as the GitHub link, deployment details, and proof sections. The appendices provide supporting information such as output screenshots and system architecture diagrams.

The document is organized in a structured manner so that the reader can easily understand the system from a general overview to detailed technical requirements.

2. General Description

This section describes the overall characteristics of the Traffic and Accident Management System. It explains the general factors that affect the system and its requirements. The purpose of this section is to provide a clear understanding of the system's working environment, features, and limitations.

This section does not define detailed or specific requirements. Instead, it gives a high-level view of the system so that the detailed requirements in the next section can be understood more easily.

The information provided here includes the system's perspective, its main functions, the type of users who will interact with it, and the constraints and assumptions under which the system operates. This helps in understanding how the system fits into a real-world scenario and what factors influence its design and performance.

2.1 Product Perspective

The Traffic and Accident Management System using Computer Vision is developed as a standalone software application. It processes traffic video data to perform automated vehicle detection, tracking, speed estimation, and accident detection.

The system uses existing technologies such as deep learning models and computer vision libraries for its implementation. It takes video input from a camera or recorded file and generates an annotated output video showing vehicle details and accident alerts. The system does not depend on any external traffic management system for its operation.

In a broader perspective, this system can be considered a part of intelligent transportation systems and smart city solutions. It can be extended in the future to integrate with real-time monitoring systems, traffic control units, or emergency response services.

This subsection provides a general view of how the system relates to similar technologies and how it can be expanded or integrated with other systems in real-world applications.

2.2 Product Functions

This subsection provides a summary of the main functions performed by the Traffic and Accident Management System using Computer Vision.

The system processes input video footage and analyzes it frame by frame to extract useful information about vehicle movement and behavior. It detects different types of vehicles such as cars, buses, trucks, and motorcycles using a deep learning model. After detection, the system tracks each vehicle across consecutive frames by assigning a unique identifier, which helps in monitoring its movement over time.

The system also estimates the speed of each vehicle by analyzing its displacement between frames. This speed is calculated and displayed in real-time on the output video. In addition, the system monitors vehicle behavior to detect abnormal patterns such as sudden stops, irregular motion, or overlapping vehicles, which may indicate an accident.

Finally, the system generates an annotated output video that includes bounding boxes around vehicles, their IDs, speed information, and accident alerts. This output helps users easily understand traffic conditions and identify critical events.

Overall, the system automates traffic analysis and provides a simple and effective way to monitor vehicle activity and detect accidents.

2.3 User Characteristics

This subsection describes the general characteristics of the users who are expected to use the Traffic and Accident Management System. These characteristics help in understanding how the system should be designed to meet user needs.

The primary users of this system include students and researchers working in the field of machine learning and computer vision, developers interested in video analytics, and basic-level traffic monitoring personnel. Most users are expected to have basic knowledge of computers and some familiarity with programming concepts, especially Python.

The system is designed to be simple and user-friendly so that even users with limited technical expertise can understand the output. The interface mainly consists of an annotated video, which makes it easy to interpret results without requiring advanced technical skills.

For advanced users such as developers and researchers, the system can also be modified or extended, as it is based on standard libraries and modular design. This flexibility allows users to improve or customize the system according to their requirements.

Overall, the system is designed to be accessible to both technical and non-technical users, ensuring ease of use while also supporting further development and experimentation.

2.4 General Constraints

This subsection describes the general limitations and constraints that may affect the design and implementation of the Traffic and Accident Management System.

The performance and accuracy of the system depend on the quality of the input video. Low-resolution videos, poor lighting conditions, or heavy occlusion between vehicles can reduce detection accuracy. The system also assumes a fixed camera position; frequent camera movement can affect tracking and speed estimation.

The system is limited by hardware capabilities. Systems with higher processing power, especially those with GPU support, can achieve better performance and faster processing. On systems with only CPU support, the processing speed may be slower.

The system relies on pre-trained deep learning models such as YOLO, which may not perform equally well in all environments without further training or tuning. Additionally, the speed estimation is based on a pixel-to-meter conversion factor, which may not be perfectly accurate without proper calibration.

The system is developed using specific software libraries such as Python, OpenCV, and YOLO, which must be properly installed and configured. Any changes or compatibility issues in these libraries may affect system performance.

These constraints should be considered during system design and testing to ensure reliable and efficient operation.

2.5 Assumptions and Dependencies

This subsection lists the assumptions and dependencies that may affect the requirements of the Traffic and Accident Management System. These factors are not direct design constraints, but any change in them may require updates to the system requirements.

The system assumes that the input video has a stable frame rate and sufficient resolution for accurate detection. It also assumes that vehicles are clearly visible in the video and that the camera remains fixed during recording. If these conditions are not met, the performance of detection, tracking, and speed estimation may be affected.

The system assumes that the required software environment is available, including Python and necessary libraries such as OpenCV and YOLO. It also assumes that the system has sufficient

computational resources to process video data. If the required environment or hardware is not available, the system may not function as expected.

The system depends on pre-trained deep learning models for object detection and tracking. Any changes in these models or their performance can directly impact the accuracy of the system. Additionally, the speed estimation depends on a predefined pixel-to-meter conversion factor, which assumes a consistent camera setup. Changes in camera angle or position may require recalibration.

These assumptions and dependencies should be considered carefully, as any variation may affect the overall functionality and accuracy of the system.

3. SPECIFIC REQUIREMENTS

This section provides a detailed description of the requirements of the Traffic and Accident Management System. These requirements define the behavior and functionality of the system and serve as a basis for system design, implementation, and testing.

The requirements are written in a clear and structured manner so that they can be easily understood and implemented. Each requirement in this section follows standard SRS quality guidelines.

Each requirement is:

- Correct: The requirement accurately represents the intended system functionality.
- Traceable: Each requirement can be traced to system objectives and future implementation.
- Unambiguous: The requirement is clearly stated and does not create confusion.
- Verifiable: The requirement can be tested and validated.
- Prioritized: Requirements are defined based on importance and stability.
- Complete: Each requirement contains all necessary details.
- Consistent: Requirements do not conflict with each other.
- Uniquely Identifiable: Each requirement is assigned a unique number for easy reference.

Special attention should be given to organizing the requirements in a structured manner so that they can be easily accessed and understood. It is also important to note that this SRS is not a software design document. Therefore, detailed design decisions should not be included, and the requirements should not overly restrict the design of the system.

This section is further divided into subsections such as external interface requirements, functional requirements, and non-functional requirements, which describe the system in detail.

3.1 External Interface Requirements

This subsection describes the external interfaces of the Traffic and Accident Management System. It defines how the system interacts with users, hardware, and software components.

These interfaces ensure proper communication between different parts of the system and help in understanding how input is provided and output is generated.

The system mainly interacts with a video input source and produces an output video with visual annotations. It also depends on specific software libraries for processing and analysis. The interface requirements are categorized into user interfaces, hardware interfaces, software interfaces, and communication interfaces.

These interfaces are designed to be simple, efficient, and easy to use so that users can operate the system without difficulty. The detailed description of each interface type is provided in the following subsections.

3.1.1 User Interfaces

The Traffic and Accident Management System provides a simple visual interface through the processed output video. The system displays the input video with additional information overlaid on each frame.

Detected vehicles are shown using rectangular bounding boxes. Each vehicle is assigned a unique ID, which is displayed above the bounding box. The system also shows the estimated speed of each vehicle in kilometres per hour (km/h).

If an abnormal situation or possible accident is detected, the system highlights the vehicle in a different color, such as red, and displays an alert label. This makes it easy for the user to identify critical events.

The interface is designed to be clear and easy to understand. It does not require complex user interaction, as the system runs automatically once the video input is provided. The output video provides all necessary information in a visual format, making it accessible even to users with basic technical knowledge.

3.1.2 Hardware Interfaces

The Traffic and Accident Management System requires basic hardware components for its operation. The primary input to the system is a video source, which can be either a live camera feed or a pre-recorded video file.

The system runs on a computer or laptop with sufficient processing capability. A system with a Graphics Processing Unit (GPU) can significantly improve performance, especially for real-time processing. However, the system can also operate on a CPU-based setup with reduced speed.

No specialized hardware devices are required, as the system is designed to function using commonly available computing resources.

3.1.3 Software Interfaces

The system is developed using Python and depends on several software libraries and tools for its implementation. The main software components include OpenCV for video processing and Ultralytics YOLO for object detection.

These libraries provide functions for reading video frames, detecting objects, and performing image processing operations. The system also uses tracking algorithms to maintain the identity of detected vehicles across frames.

The software environment should support the installation and execution of these libraries. Compatibility between library versions is important for proper system functioning.

3.1.4 Communication Interfaces

The system does not require any external communication with other systems or networks. It operates independently and processes video data locally on the system.

There is no requirement for data transfer over the internet or interaction with remote servers. However, the system can be extended in the future to support communication features such as cloud storage or real-time data sharing if needed.

3.2 Functional Requirements

This section describes the main functional requirements of the Traffic and Accident Management System. These requirements define the specific features and operations that the system must perform.

Each functional requirement is described clearly so that it can be implemented and tested. The system processes video input and performs a sequence of operations including vehicle detection, tracking, speed estimation, and accident detection.

These requirements ensure that the system behaves as expected and provides accurate results. Each function is essential for achieving the overall objective of automated traffic monitoring and accident detection.

The detailed functional requirements are described in the following subsections.

3.2.1 Vehicle Detection

3.2.1.1 Introduction

This requirement describes how the system identifies vehicles in each video frame. The system uses a deep learning model to detect objects and classify them as vehicles such as cars, buses, trucks, and motorcycles.

3.2.1.2 Inputs

- Video frames obtained from the input video source
- Pre-trained object detection model (YOLO)

3.2.1.3 Processing

Each frame is passed to the object detection model. The model analyzes the frame and identifies objects that belong to vehicle classes. For each detected vehicle, the model generates a bounding box along with a confidence score. Only detections above a certain confidence threshold are considered valid.

3.2.1.4 Outputs

The system generates the following outputs after processing each video frame:

- Bounding boxes around detected vehicles
- Class labels indicating the type of vehicle (car, bus, truck, motorcycle)
- Confidence scores for each detected object

These outputs are displayed on the processed video, making it easy for the user to identify detected vehicles.

3.2.1.5 Error Handling

If the system fails to detect vehicles due to poor video quality, low lighting, or occlusion, those detections are ignored. The system continues processing the next frames without interruption. Low-confidence detections are filtered out to maintain accuracy.

3.2.2 Vehicle Tracking

3.2.2.1 Introduction

This requirement describes how the system tracks detected vehicles across frames by assigning unique IDs.

3.2.2.2 Inputs

- Detected bounding boxes
- Video frames

3.2.2.3 Processing

The system assigns a unique ID to each detected vehicle and tracks its movement across frames using a tracking algorithm. This helps maintain continuity of each vehicle.

3.2.2.4 Outputs

- Unique ID for each vehicle
- Updated position of vehicles across frames

3.2.2.5 Error Handling

If tracking is lost due to occlusion or detection failure, the system attempts to reassign the same ID. If not possible, a new ID is assigned.

3.5 Non-Functional Requirements

This section describes the non-functional requirements of the Traffic and Accident Management System. These requirements define the quality attributes and performance constraints of the system. They ensure that the system operates efficiently, reliably, and consistently under different conditions.

The non-functional requirements are specified in measurable terms so that they can be evaluated and tested.

3.5.1 Performance

The system should process video frames efficiently to provide near real-time results. It should process at least 20–30 frames per second on a standard system. The system should detect vehicles and display results within 1 second of processing each frame. It should support video resolutions up to 1080p without significant delay.

3.5.3 Availability

The system should remain available during execution without unexpected interruptions. It should operate continuously while processing video input. System downtime should not exceed 1 minute during continuous operation.

3.5.4 Security

The system should not store or process any sensitive personal data. All video processing should be performed locally on the system. The system should ensure that no data is shared with external servers unless explicitly required in future enhancements.

3.5.5 Maintainability

The system should be designed in a modular and structured manner to allow easy updates and modifications. The code should be well-organized and documented so that future developers can maintain and improve the system efficiently.

3.5.6 Portability

The system should be compatible with multiple platforms such as Windows and Linux. It should also be able to run on cloud-based platforms such as Google Colab and Kaggle with minimal configuration changes.

3.7 Design Constraints

The design and implementation of the Traffic and Accident Management System are subject to certain constraints.

The system relies on pre-trained deep learning models such as YOLO, which limits flexibility in detection performance. The accuracy of the system depends on video quality, lighting conditions, and camera angle. A fixed camera setup is required for proper tracking and speed estimation.

The system depends on specific software libraries such as Python, OpenCV, and YOLO. Proper installation and compatibility of these libraries are necessary for system operation.

Hardware limitations such as lack of GPU support may affect processing speed and performance. The system must be designed to work within these limitations while maintaining acceptable performance.

3.9 Other Requirements

This section includes additional requirements that are not covered in the previous sections but are important for the proper functioning of the system.

The system should automatically save the processed output video in the specified output directory without requiring manual intervention. The output should include all annotations such as bounding boxes, vehicle IDs, speed information, and accident alerts.

The system should be easy to use, with minimal user interaction required. The user should only need to provide the input video, and the system should handle the processing automatically.

The system should handle errors gracefully without stopping execution. In case of minor issues such as temporary detection failure, the system should continue processing the remaining frames.

Proper documentation should be provided so that users can understand how to run and modify the system. This will help in future improvements and maintenance.

4. ANALYSIS MODELS

This section presents the analysis models used to understand the system requirements described in Section 3. These models show how data flows through the system and how different components interact with each other.

Each model includes a brief introduction and description. The models are directly related to the functional requirements such as vehicle detection, vehicle tracking, speed estimation, and accident detection. This ensures proper traceability between the system design and requirements.

4.1 Data Flow Diagrams (DFD)

5. GitHub link: https://github.com/Shahid332005/accidentdetection

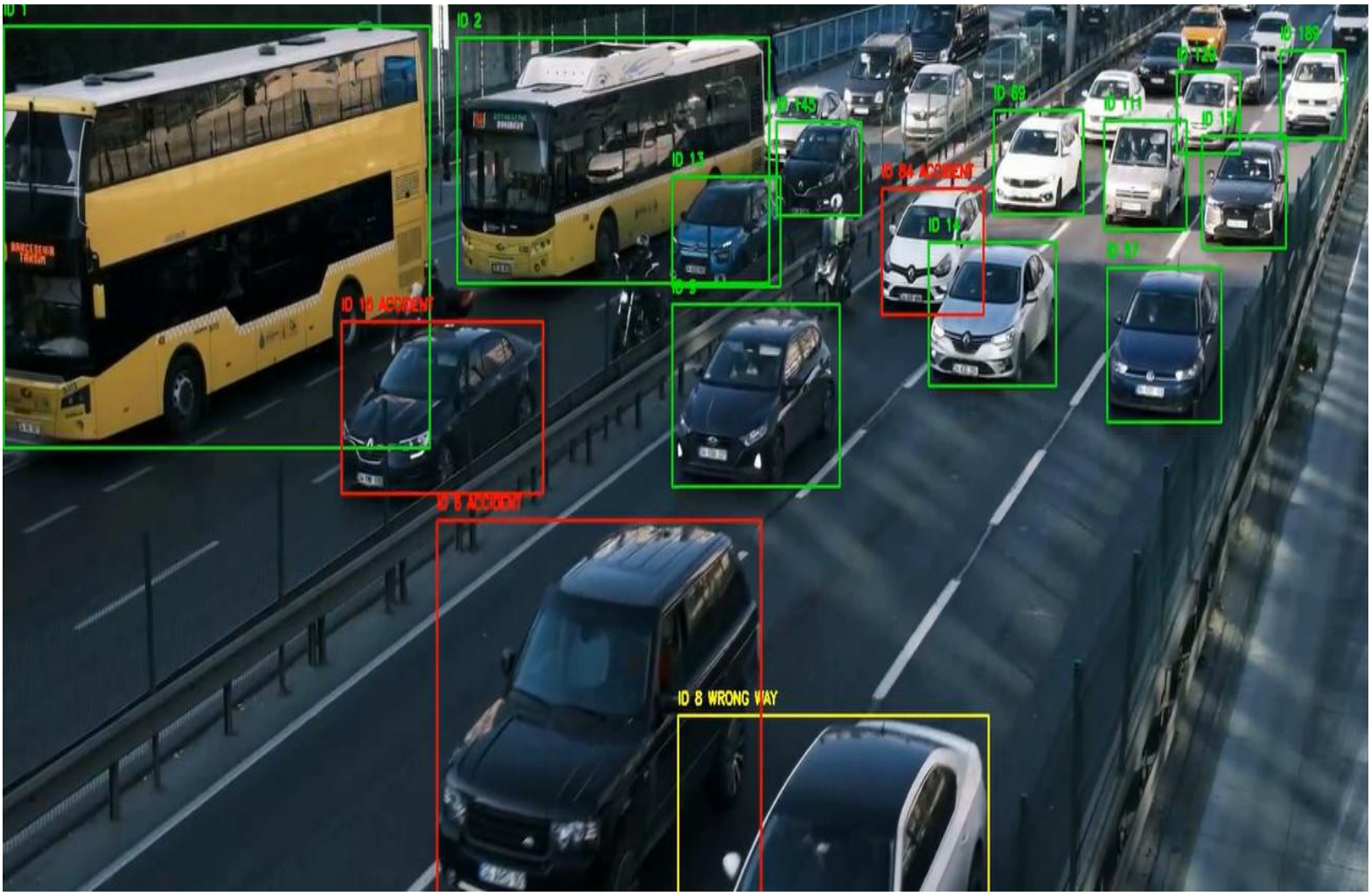
A. Appendices

This section provides supplementary information that supports the Software Requirements Specification (SRS) for the Traffic and Accident Management System using Computer Vision.

The information presented in this section is intended to enhance understanding of the system design, implementation, and operation. Unless explicitly stated, the appendices are not considered part of the core system requirements**.**

A.1 Appendix 1: System Workflow and Output

The system follows a structured pipeline to process traffic video data and detect accidents.



This appendix presents the output generated by the Traffic and Accident Management System. The system processes the input video and detects multiple vehicles in real-time.

In the output, vehicles are highlighted using bounding boxes. Each detected vehicle is assigned a unique ID, which helps in tracking the vehicle across frames. The system also identifies abnormal events such as accidents and wrong-way driving.

Vehicles marked in **green** represent normal detected vehicles. Vehicles highlighted in **red** indicate accident detection, where abnormal behavior such as sudden collision or stop is observed. Additionally, vehicles marked in **yellow** indicate wrong-way driving, which is considered a traffic violation.

The system successfully demonstrates multiple functionalities including:

- Real-time vehicle detection
- Vehicle tracking using unique IDs
- Accident detection
- Wrong-way detection

This output verifies that the system is working correctly and is capable of analyzing real-world traffic scenarios.

Workflow Description:

- The system accepts video input from a camera or file
- Each frame is processed using a deep learning model (YOLO)
- Detected vehicles are assigned unique IDs for tracking
- Movement between frames is analyzed to calculate speed
- Abnormal behavior is detected to identify accidents
- Final output is generated as an annotated video

This workflow ensures efficient and automated traffic monitoring.

A.2 Appendix 2 – Tools and Technologies

This appendix lists the tools, technologies, and platforms used to develop and execute the Traffic and Accident Management System. These components enable video processing, vehicle detection, tracking, speed estimation, and accident identification.

Development Environment

- Python 3.x

Libraries and Frameworks

- OpenCV – for video capture and frame processing
- Ultralytics YOLOv8 – for real-time vehicle detection and tracking
- NumPy – for numerical computations and array operations

Execution Platforms

- Local System (CPU/GPU)
- Google Colab
- Kaggle Notebook (optional)

Dataset

- Traffic video datasets (publicly available or self-recorded)
- Example: highway traffic videos with multiple vehicles and stable camera view

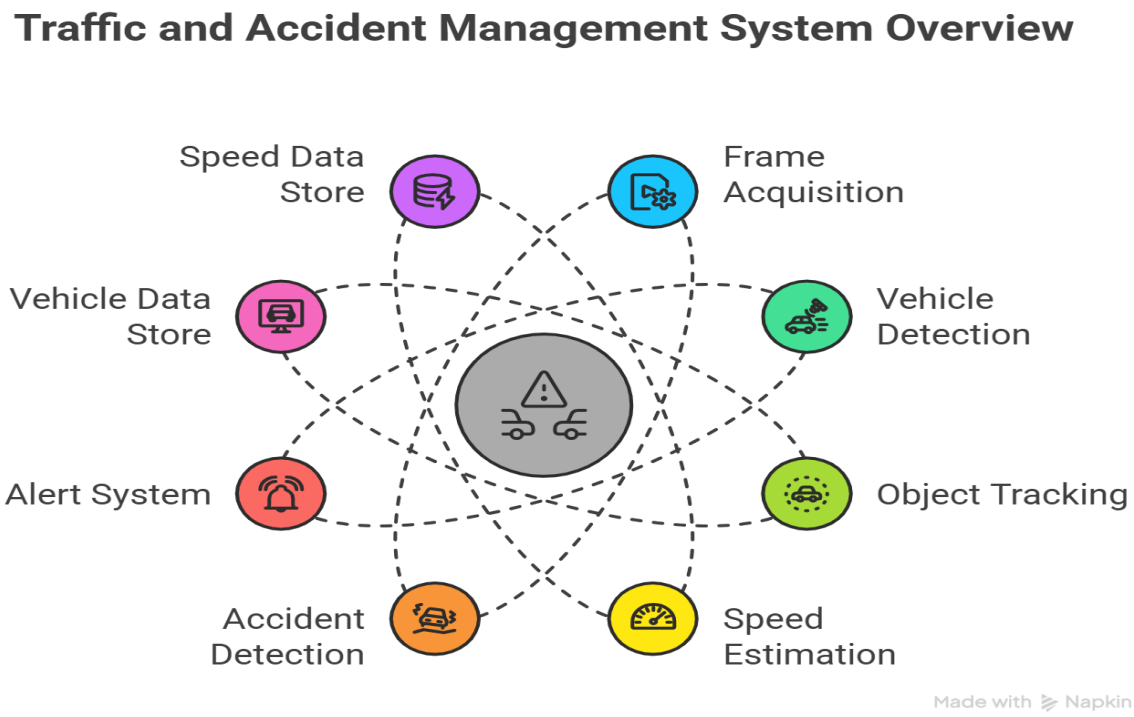
---

Output Generated

The system generates a processed video with the following features:

- Bounding boxes around detected vehicles
- Unique vehicle IDs for tracking
- Estimated speed (km/h)
- Accident alerts for abnormal events (highlighted in red)

Wrong-way detection (highlighted in yellow, if applicable)



System Architecture Diagram Description:

The above diagram represents the overall architecture of the Traffic and Accident Management System. It shows how different components of the system are interconnected and how data flows between them.

The process begins with frame acquisition, where video frames are captured from the input source. These frames are then passed to the vehicle detection module, which identifies vehicles present in each frame using deep learning techniques.

After detection, the object tracking module assigns unique IDs to each vehicle and tracks their movement across frames. The speed estimation module calculates the speed of each vehicle based on its displacement over time.

The accident detection module analyzes vehicle behavior and identifies abnormal events such as sudden stops or collisions. In such cases, the alert system generates notifications for potential accidents.

Additionally, the system stores important information such as vehicle data and speed data for further analysis. This architecture ensures smooth data flow and efficient processing of traffic video streams.

Remarks:

The selected tools and technologies ensure efficient processing, scalability, and ease of implementation for computer vision-based traffic analysis.